BACKEND ENGINEERING

DAY 7

Connecting FastAPI with MongoDB

A practical guide to MongoDB, PyMongo, database connections, collections, ObjectId serialization, Pydantic schemas, and complete CRUD APIs.

Your learning sequence
Day 5: Relational database and SQL fundamentals
Day 6: FastAPI + PostgreSQL
Day 7: FastAPI + MongoDB — your main NoSQL database track

1. What you will learn

• Understand how FastAPI communicates with MongoDB.

• Understand database, collection, document, and field.

• Connect using PyMongo and a MongoDB connection string.

• Keep database configuration separate from routes.

• Understand why MongoDB uses ObjectId and why it must be serialized for JSON.

• Use Pydantic schemas for request validation and response formatting.

• Build POST, GET, PUT, and DELETE endpoints.

• Handle invalid IDs, duplicate UIDs, missing documents, and database errors.

Prerequisites: FastAPI routes, Pydantic BaseModel, Python dictionaries/lists, HTTP methods, and your previous JSON CRUD practice.

2. MongoDB concepts

Database: a container that holds collections. **Collection:** a group of related documents, similar in purpose to a SQL table. **Document:** one BSON record, usually represented like a JSON object. **Field:** one key/value inside a document.

```
MongoDB server
    |
    +-- hospital_db (database)
           |
           +-- doctors (collection)
                  |
                  +-- document 1: { uid, name, age, dept }
                  +-- document 2: { uid, name, age, dept }
```

MongoDB is document-oriented. Documents in one collection can have different fields, but your application should still define a consistent data shape and validate incoming data.

3. PyMongo: the Python driver

PyMongo is the official MongoDB Python driver. In this lesson, use its synchronous MongoClient with normal synchronous FastAPI route functions (def). This keeps the first connection exercise straightforward. Do not call synchronous database operations directly inside async def routes, because blocking work can block the event loop.

```
python -m pip install fastapi uvicorn pymongo python-dotenv
```

Verify that the packages are installed in the same virtual environment used to run Uvicorn:

```
python -m pip show fastapi pymongo python-dotenv
```

4. Create or access your MongoDB database

You can use MongoDB Atlas (cloud) or a local MongoDB server. In Atlas, create a database user, allow the appropriate network access, and copy the Python driver connection string. Do not share your real connection string or password.

Create a .env file:

```
MONGO_URI=mongodb+srv://USERNAME:PASSWORD@YOUR_CLUSTER.mongodb.net/?retryWrites=true&w=majority
```

Replace the placeholders with your own Atlas details. If your password contains reserved URL characters such as @, :, /, ?, #, or %, URL-encode the username/password components before placing them in the URI. Keep .env out of Git.

```
.gitignore\n.env\n.venv/\n__pycache__/
```

5. Project structure

```
fastapi_mongodb/
|-- main.py
|-- database.py
|-- schemas.py
|-- .env
|-- .gitignore
```

For this lesson, keep the project simple. Later, you can separate routes into routers/ and business logic into services/.

6. Connect FastAPI to MongoDB — database.py

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient
from pymongo.server_api import ServerApi

load_dotenv()

MONGO_URI = os.getenv("MONGO_URI")
if not MONGO_URI:
    raise RuntimeError("MONGO_URI is not configured")

client = MongoClient(MONGO_URI, server_api=ServerApi("1"))

try:
    client.admin.command("ping")
    print("Connected to MongoDB successfully")
except Exception as error:
    print("MongoDB connection failed:", error)
    raise

database = client["hospital_db"]
doctor_collection = database["doctors"]
```

Understand each line:

• MongoClient(MONGO_URI) creates the client used to communicate with MongoDB.

• The ping command checks that the server can be reached and authenticated.

• client['hospital_db'] selects the database.

• database['doctors'] selects the collection. MongoDB creates the database/collection when data is first written if they do not already exist.

• A collection variable is not a list or Python dictionary; it is a PyMongo collection handle used to run database operations.

Important: MongoClient manages a connection pool. In a larger application, manage its lifecycle with FastAPI lifespan and close it at shutdown. This first lesson keeps the setup visible and easy to follow.

7. Understand MongoDB ObjectId

MongoDB automatically creates an _id field when a document is inserted without one. In a typical collection, this is an ObjectId. ObjectId is not a normal string, so convert it to str before returning it as JSON.

```
from bson import ObjectId

document = {
    "_id": ObjectId("65a1b2c3d4e5f67890123456"),
    "name": "Ravi"
}

# JSON-friendly representation:
{
    "id": "65a1b2c3d4e5f67890123456",
    "name": "Ravi"
}
```

The MongoDB field is named _id; your API can expose it as id. This is the same serialization idea you previously practised with your indi_serial() function.

8. Create Pydantic schemas — schemas.py

```
from pydantic import BaseModel, Field, ConfigDict

class DoctorCreate(BaseModel):
    uid: str = Field(min_length=3, max_length=20)
    name: str = Field(min_length=1, max_length=20)
    age: int = Field(gt=0)
    dept: str
    branch: str
    phno: str
    address: str

class DoctorUpdate(BaseModel):
    name: str = Field(min_length=1, max_length=20)
    age: int = Field(gt=0)
    dept: str
    branch: str
    phno: str
    address: str
```

```
class DoctorResponse(BaseModel):
    id: str
    uid: str
    name: str
    age: int
    dept: str
    branch: str
    phno: str
    address: str
```

DoctorCreate validates the incoming POST body. DoctorUpdate validates the full PUT body. DoctorResponse defines exactly which fields are allowed in the API response. Do not include passwords or secret fields in a public response schema.

9. Serialization functions — main.py

Create a function to convert one MongoDB document into a JSON-friendly dictionary. Then create a function for a list of documents:

```
def doctor_serial(doctor) -> dict:
    return {
        "id": str(doctor["_id"]),
        "uid": doctor["uid"],
        "name": doctor["name"],
        "age": doctor["age"],
        "dept": doctor["dept"],
        "branch": doctor["branch"],
        "phno": doctor["phno"],
        "address": doctor["address"]
    }

def list_of_doctors_serial(doctors) -> list:
    return [doctor_serial(doctor) for doctor in doctors]
```

Why do we need this? MongoDB returns _id as ObjectId. FastAPI's JSON response needs a JSON-compatible value. This function also controls which database fields are exposed.

10. Complete CRUD API — main.py

The following is a complete beginner-friendly example. It uses a string uid as a unique business identifier and MongoDB's ObjectId for the document URL.

```
from fastapi import FastAPI, HTTPException
from pymongo.errors import DuplicateKeyError
from bson import ObjectId
from database import doctor_collection, client
from schemas import DoctorCreate, DoctorUpdate, DoctorResponse

app = FastAPI()

# Ensure each doctor UID is unique.
doctor_collection.create_index("uid", unique=True)


def doctor_serial(doctor) -> dict:
    return {
        "id": str(doctor["_id"]),
        "uid": doctor["uid"],
        "name": doctor["name"],
        "age": doctor["age"],
        "dept": doctor["dept"],
        "branch": doctor["branch"],
        "phno": doctor["phno"],
        "address": doctor["address"]
    }


def list_of_doctors_serial(doctors) -> list:
    return [doctor_serial(doctor) for doctor in doctors]


@app.post("/doctors", response_model=DoctorResponse, status_code=201)
def create_doctor(doctor: DoctorCreate):
    try:
        result = doctor_collection.insert_one(doctor.model_dump())
    except DuplicateKeyError:
        raise HTTPException(status_code=409, detail="Doctor UID already exists")

    saved = doctor_collection.find_one({"_id": result.inserted_id})
    return doctor_serial(saved)


@app.get("/doctors", response_model=list[DoctorResponse])
def get_all_doctors():
    doctors = doctor_collection.find()
```

```
    return list_of_doctors_serial(doctors)


@app.get("/doctors/{doctor_id}", response_model=DoctorResponse)
def get_doctor(doctor_id: str):
    if not ObjectId.is_valid(doctor_id):
        raise HTTPException(status_code=400, detail="Invalid doctor ID")

    doctor = doctor_collection.find_one({"_id": ObjectId(doctor_id)})
    if doctor is None:
        raise HTTPException(status_code=404, detail="Doctor not found")

    return doctor_serial(doctor)


@app.put("/doctors/{doctor_id}", response_model=DoctorResponse)
def update_doctor(doctor_id: str, data: DoctorUpdate):
    if not ObjectId.is_valid(doctor_id):
        raise HTTPException(status_code=400, detail="Invalid doctor ID")

    result = doctor_collection.update_one(
        {"_id": ObjectId(doctor_id)},
        {"$set": data.model_dump()}
    )

    if result.matched_count == 0:
        raise HTTPException(status_code=404, detail="Doctor not found")

    updated = doctor_collection.find_one({"_id": ObjectId(doctor_id)})
    return doctor_serial(updated)


@app.delete("/doctors/{doctor_id}")
def delete_doctor(doctor_id: str):
    if not ObjectId.is_valid(doctor_id):
        raise HTTPException(status_code=400, detail="Invalid doctor ID")

    result = doctor_collection.delete_one({"_id": ObjectId(doctor_id)})

    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Doctor not found")

    return {"message": "Doctor deleted successfully"}


@app.on_event("shutdown")
def close_mongodb():
    client.close()
```

Note about PUT: this example accepts all editable fields and uses $set to replace those fields' values. Fields not included in the update schema, such as uid, remain unchanged. For partial updates, create a PATCH endpoint with optional fields.

11. Understand the MongoDB CRUD methods

PyMongo method	Purpose
insert_one(document)	Insert one document
find_one(filter)	Find one matching document
find(filter)	Find matching documents
update_one(filter, update)	Update one matching document
delete_one(filter)	Delete one matching document
create_index(field, unique=True)	Create a unique index

12. Test using Swagger UI

```
uvicorn main:app --reload
```

Open http://127.0.0.1:8000/docs. Test these endpoints:

• POST /doctors — create a doctor.

• GET /doctors — display all doctors.

• GET /doctors/{doctor_id} — copy the id returned by POST and retrieve that doctor.

• PUT /doctors/{doctor_id} — send all editable fields to update the record.

• DELETE /doctors/{doctor_id} — delete the record.

• GET the same ID again — expect HTTP 404.

Sample POST request body:

```
{
  "uid": "DOC101",
  "name": "Ravi",
  "age": 35,
  "dept": "Cardiology",
  "branch": "Main",
  "phno": "9876543210",
  "address": "Guntur"
}
```

13. Common errors and debugging

• ModuleNotFoundError: activate the correct virtual environment and install PyMongo in that environment.

• Server selection timeout: check internet access, Atlas network access/IP allowlist, cluster status, and URI host.

• Authentication failed: verify the database username/password; do not confuse your Atlas website login with the database user.

• Invalid URI: check the connection string and URL-encode reserved characters in credentials.

• Invalid ObjectId: check that the path ID is a valid 24-character hexadecimal ObjectId.

• Object of type ObjectId is not JSON serializable: convert _id to str before returning it.

• DuplicateKeyError: the unique uid already exists; choose a different UID.

• 404 Not Found: verify the ID exists and the correct database/collection is selected.

• 500 Internal Server Error: read the full terminal traceback and identify the failing line.

14. MongoDB vs PostgreSQL recap

Concept	MongoDB	PostgreSQL
Structure	Database → collection → document	Database → table → row
Identifier	Usually _id / ObjectId	Primary key column
Data access	PyMongo methods and filters	SQL / SQLAlchemy
Update example	$set operator	UPDATE statement
Schema approach	Flexible documents; validate in app	Defined table columns and constraints

Neither database is automatically better for every project. Choose based on data relationships, consistency needs, query patterns, and the team's requirements.

15. Security and good practices

• Keep the MongoDB URI in environment variables and never commit .env.

• Use a least-privilege database user rather than an administrator account.

• Restrict Atlas network access to trusted IPs or approved network paths.

• Never return passwords, tokens, or internal-only fields.

• Validate request data with Pydantic and enforce uniqueness with a database index.

• For production, manage client lifecycle with FastAPI lifespan, add logging, tests, authentication, authorization, and careful error handling.

• Do not expose database exception details or credentials to API clients.

16. Practical assignment

• Create a MongoDB database named hospital_db and a collection named doctors.

• Connect your FastAPI project using database.py.

• Create DoctorCreate, DoctorUpdate, and DoctorResponse schemas.

• Implement all five endpoints: create, get all, get one, update, and delete.

• Create two doctors with different UIDs.

• Try inserting the same UID twice and confirm the API returns 409.

• Try an invalid ObjectId and confirm the API returns 400.

• Delete a doctor and confirm a later GET returns 404.

• Explain your doctor_serial() function line by line.

17. Review questions

• What is the difference between a database, collection, and document?

• What is PyMongo?

• What does MongoClient do?

• Why do we run the ping command?

• What is ObjectId, and why do we convert it to str?

• What is the difference between find_one() and find()?

• What does $set do in update_one()?

• Why create a unique index on uid?

• Why do we need Pydantic schemas when MongoDB is schema-flexible?

• Why should synchronous PyMongo calls be used with normal def routes in this lesson?

18. Completion checklist

• ■ I understand database, collection, document, and field.

• ■ I installed PyMongo in my project's virtual environment.

• ■ I connected FastAPI to MongoDB using an environment variable.

• ■ I understand MongoClient, database, and collection.

• ■ I understand ObjectId and JSON serialization.

• ■ I created Pydantic request and response schemas.

• ■ I implemented create, read, update, and delete endpoints.

• ■ I tested duplicate UID, invalid ID, and missing document cases.

• ■ I can explain my serialization function without memorizing it.

19. Official documentation

• MongoDB PyMongo documentation: https://www.mongodb.com/docs/languages/python/pymongo-driver/current/

• MongoDB connection guide: https://www.mongodb.com/docs/languages/python/pymongo-driver/current/connect/

• MongoDB CRUD operations: https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/

• FastAPI documentation: https://fastapi.tiangolo.com/

• Pydantic models: https://docs.pydantic.dev/latest/concepts/models/

Suggested Day 8: Build a small Hospital Management API using MongoDB with separate routers, config, schemas, serialization utilities, and environment-based settings. Add search by department and pagination.